

alphai

L'ÉDUCATION À L'INTELLIGENCE ARTIFICIELLE

Algorithme des k plus proches voisins

Introduction

Cette activité présente un algorithme d'apprentissage relativement simple appelé l'algorithme des k plus proches voisins. L'abréviation de cet algorithme est KNN, qui vient de l'anglais *k nearest neighbors*.

Voici les trois objectifs de l'activité, correspondant approximativement aux trois parties qui le composent :

1. Comprendre la visualisation de l'espace d'états 2D présente dans le logiciel **AlphAI**.
2. Comprendre le principe de l'algorithme des k plus proches voisins.
3. Être capable de programmer soi-même cet algorithme en python.

NB : les captures d'écran du logiciel **AlphAI** présentées dans ce document sont issues de la version 1.9.3.5. Votre écran pourra donc être légèrement différent si vous utilisez une autre version du logiciel.

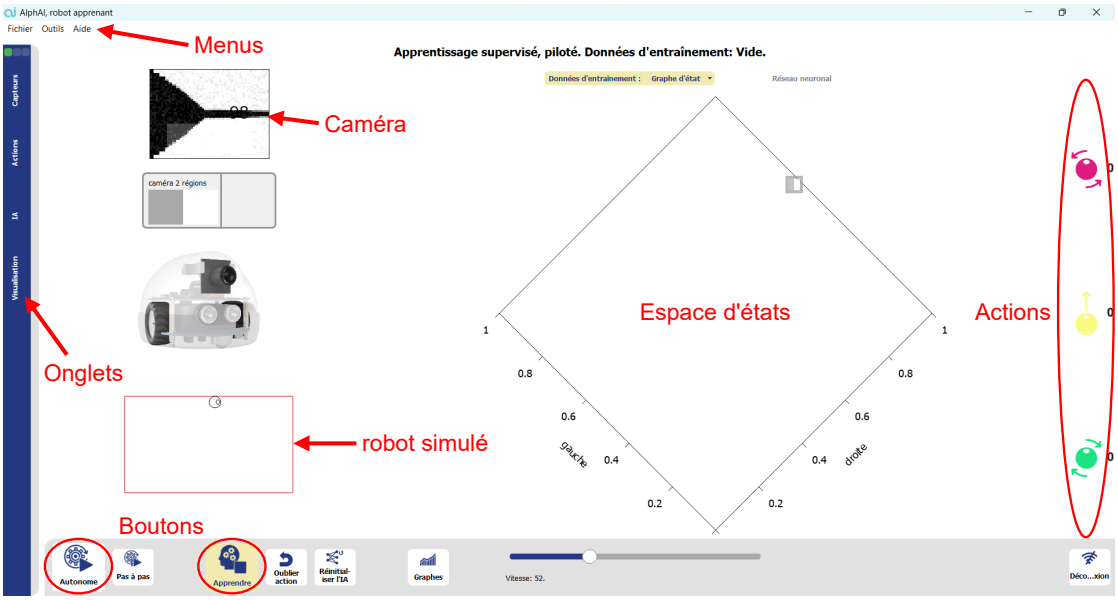


FIGURE 1 – Écran principal de **AlphAI** dans le scénario KNN

1 PROGRAMMER LE ROBOT EN PYTHON

1.1 MISE EN PLACE

1. Lancez le logiciel **AlphAI**.
2. Connectez-vous à un robot ou un simulateur.
3. Dans les menus, sélectionnez **Fichier > Configurations d'exemple**, puis choisissez le premier scénario de la troisième ligne : **KNN Caméra**.
4. Dans l'onglet **Capteurs** sur le côté gauche, vous pouvez vérifier que le seul capteur actif est la caméra, et l'image apparaissant à l'écran doit être en noir et blanc.
5. Dans l'onglet **Actions**, vous pouvez vérifier que seules les actions **avancer** et **pivoter** (droite + gauche) sont actives, ce qui donne un total de 3 actions disponibles pour le robot. Les icônes d'actions sont associées à des couleurs sur la droite de l'écran. Par la suite, vous pourrez décider de remplacer les actions **pivoter** par **pivoter un peu** si vous constatez que cela améliore le comportement du robot, mais veillez bien à toujours avoir uniquement 3 actions disponibles.

1.2 COMPRENDRE LE TRAITEMENT D'IMAGE

1. Désactivez l'apprentissage en cliquant sur le bouton **apprentissage** ou en utilisant le raccourci `L` (si nécessaire, activez les raccourcis clavier à partir du menu **Fichier > Raccourcis clavier**).
2. Sur l'image de la caméra apparaissant à l'écran, vous pouvez voir deux zones

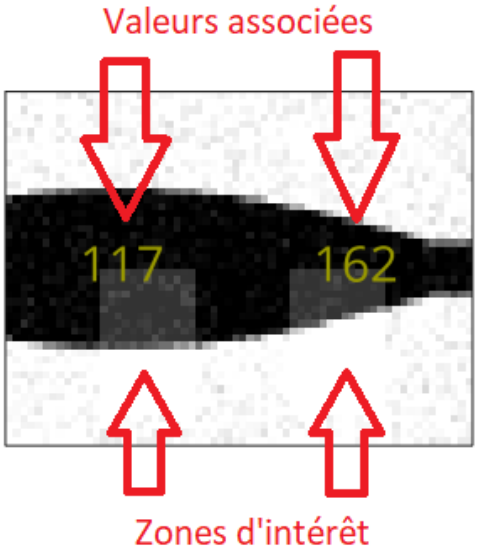


FIGURE 2 – Zones d'intérêt de l'image caméra

plus claires et deux nombres. En plaçant le robot à différents endroits, plus ou moins près des murs de l'arène, déterminez comment ces nombres sont calculés (à partir de ces deux zones de l'image).

3. Au centre de l'écran est présent un graphique sur lequel se déplace un point. En plaçant le robot à différents endroits, déterminez comment la position du point est affectée par les deux valeurs issues de l'image.
4. Indiquez comment placer le robot de manière que ce point se rapproche de chacun des 4 coins du graphique.

1.3 VISUALISER LE PROGRAMME DU ROBOT

1. Dans l'onglet **IA**, changez le paramètre **algorithme** et sélectionnez **code python**. Créez un nouveau fichier, enregistrez-le à l'emplacement de votre choix, puis ouvrez-le avec un éditeur de texte, par exemple Notepad++. **Attention :** il n'est pas nécessaire d'**exécuter** ce fichier à l'aide d'un interpréteur python (cela ne donnera rien), car les modifications apportées au fichier sont automatiquement prises en compte dans le logiciel **AlphAI** dès qu'elles sont enregistrées. Dans ce mode d'utilisation, il suffit donc de modifier le fichier et d'enregistrer ces modifications.
2. Nous allons maintenant programmer le robot depuis ce fichier python en modifiant la troisième fonction, appelée `take_decision`. Il ne faudra pas oublier d'enregistrer le fichier python après chaque modification, afin qu'elles soient bien prises en compte par le logiciel !
3. Dans un premier temps, observez les messages affichés par l'instruction `print` dans la console **AlphAI**(la fenêtre noire qui s'ouvre au lancement du logiciel). Il s'agit des valeurs du paramètre `sensors`, qui correspondent aux

valeurs mesurées par les capteurs du robot. Quelle opération mathématique permet d'obtenir les valeurs affichées dans la console à partir des valeurs affichés sur l'image de la caméra dans la fenêtre du logiciel ?

4. Dans le logiciel **AlphAI**, activez le mode **autonome** (`barre espace`). Quelle est l'action choisie par le robot ?
5. Dans le fichier python, modifiez la valeur renvoyée par la fonction `take_decision`, enregistrez la modification, puis observez l'effet sur le comportement du robot. Déterminez quelle valeur de retour correspond à chaque action disponible.
6. Vous devriez maintenant avoir compris comment modifier la fonction `take_decision` afin de programmer le robot. Essayez de programmer le comportement suivant : le robot doit aller tout droit dès que possible, mais doit éviter de toucher les parois de l'arène. **Indice :** utilisez une structure conditionnelle (`if / elif / else`).
7. Dans le logiciel **AlphAI**, observez le déplacement et les changements de couleur du point dans l'**espace d'états** (le graphique central). Dans l'onglet **Visualisation**, activez le paramètre **arrière-plan**. À quoi correspondent les différentes zones de couleur ?
8. Quelle serait la coloration idéale pour le comportement du robot souhaitée ? Essayez d'obtenir cette coloration en améliorant votre fonction `take_decision`.

2 Algorithme des k plus proches voisins

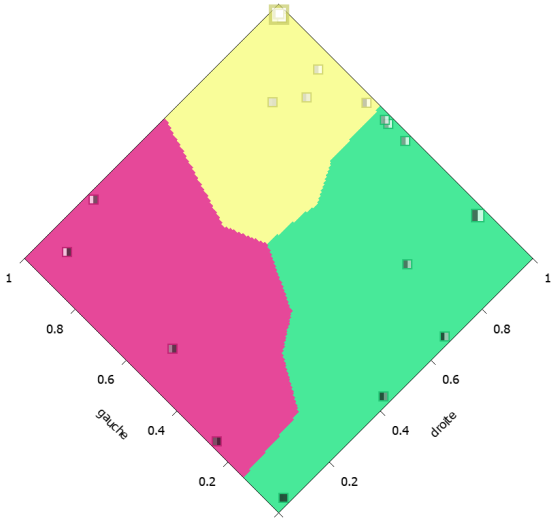
2.1 Mise en place

1. Dans cette partie, nous n'utiliserons pas le mode **code python**, vous pouvez donc sauvegarder et fermer le fichier python ouvert précédemment.
2. Dans l'onglet **IA**, sélectionnez l'algorithme **k plus proches voisins**.
3. Dans l'onglet **Visualisation**, désélectionnez le paramètre **arrière-plan**.
4. Réactivez le mode apprentissage.
5. Si vous n'êtes pas sûr que votre paramétrage est correct, vous pouvez charger de nouveau les paramètres d'exemple **KNN - Caméra**, afin de revenir aux paramètres de départ.

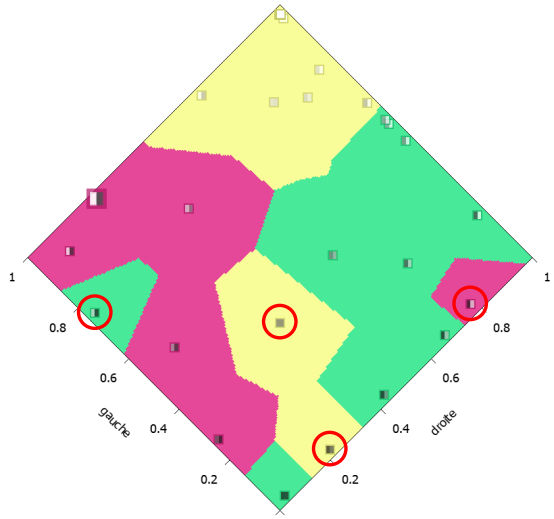
2.2 Apprentissage supervisé

L'algorithme des k plus proches voisins est un algorithme d'apprentissage supervisé, ce qui signifie que l'IA doit être entraînée à partir de **données étiquetées** fournies par des humains. Dans le cas du robot **AlphAI**, on fournit ces données simplement en pilotant le robot.

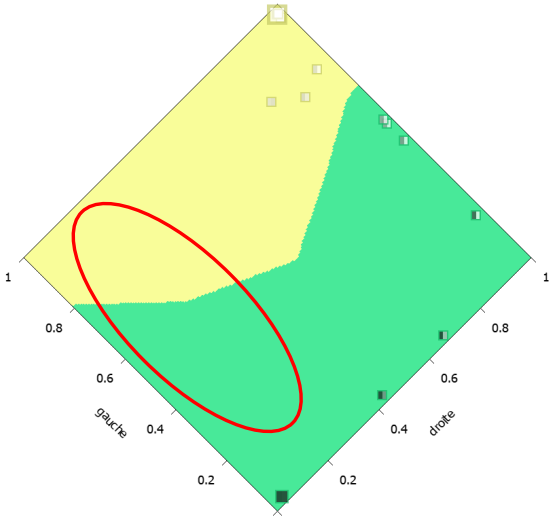
1. Assurez-vous que le bouton **Apprendre** est activé.
2. Pilotez le robot dans l'arène en évitant les murs, jusqu'à avoir environ une centaine de points dans la mémoire d'expérience (voir le statut en haut de l'écran). Puis activez le mode autonome. Le comportement du robot est-il satisfaisant ?
3. Au cours de l'apprentissage, des points (plus petits) sont apparus sur le graphique de l'espace d'états. Comment sont déterminés la position et la couleur de chaque point ?
4. Réinitialisez l'apprentissage, puis placez un point jaune (tout droit) et un point vert (pivoter à droite) dans deux coins opposés de l'espace d'états. Sans ajouter de nouveaux points (vous pouvez par exemple désactiver l'apprentissage), placez le robot dans différentes situations et observez le changement de couleur du point principal. Comment le robot prend-il sa décision ?
5. Activez le paramètre **arrière-plan** afin de vérifier votre hypothèse. Puis refaites un apprentissage de votre robot en prenant soin de bien choisir chaque action, afin d'obtenir des zones de couleur distinctes et des frontières nettes : voir les exemples à la figure 3. Utilisez le bouton **Oublier action** en cas de besoin. Le comportement du robot est-il plus satisfaisant ?
6. En conclusion : qu'est-ce qui est le plus important pour un bon apprentissage : la quantité de données d'entraînement ou la qualité de ces données (absence de biais) ?



(a) Exemple d'apprentissage cohérent et efficace : les séparations entre les différentes zones de couleur (zones de décision) sont nettes et bien définies.



(b) Exemple de **biais explicites** : les points entourés en rouge sont mal étiquetés et découpent les zones de décision. Il faudrait les supprimer ou corriger leur étiquette.



(c) Exemple de **biais implicite** : la zone entourée en rouge ne contient aucune donnée d'entraînement ; les décisions y sont arbitraires.

FIGURE 3 – Exemples de graphes d'états : (a) un bon apprentissage, (b) un apprentissage contenant des erreurs (**biais explicites**) et (c) un apprentissage incomplet (**biais implicite**).

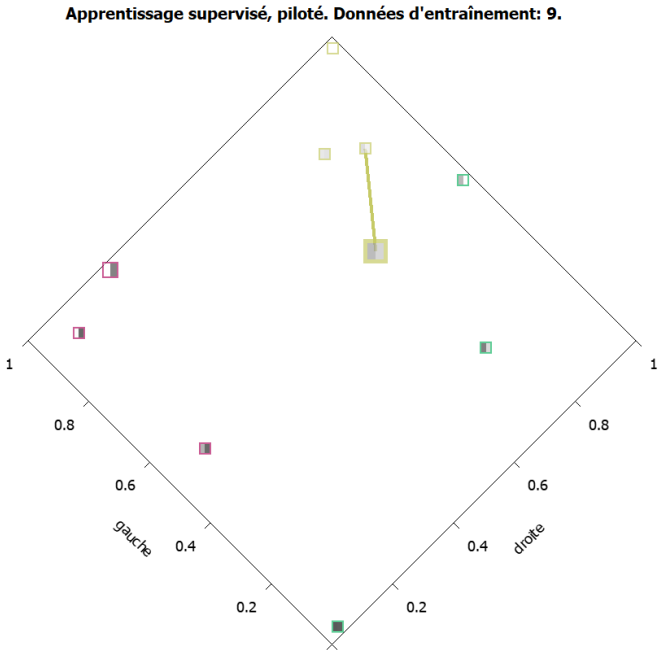


FIGURE 4 – L'espace d'états après un apprentissage avec 9 points bien séparés

2.3 LE PARAMÈTRE k

Dans la section précédente, nous avons étudié l'algorithme des k plus proches voisins avec $k = 1$. Nous allons maintenant nous intéresser à l'impact de ce paramètre k : le nombre de voisins.

1. Désactivez la coloration de l'arrière-plan et réalisez un apprentissage avec peu de points (une dizaine environ), répartis le plus uniformément possible sur le graphique de l'espace d'états (n'ajouter un nouveau point que s'il est suffisamment éloigné des autres, voir figure 4). Dans l'onglet **IA**, augmentez progressivement la valeur du paramètre k et observez l'effet sur les prises de décision du robot. Expliquez pourquoi on choisit généralement une valeur impaire pour k.
2. Modifiez de nouveau la valeur du paramètre k, mais cette fois avec la coloration de l'arrière-plan active. Observez l'effet sur les zones de décision.
3. Réalisez maintenant un entraînement du robot avec une centaine de points, et contenant un petit nombre d'erreurs : entre 5% et 10% des points ne sont pas de la bonne couleur. Quelle peut être l'utilité d'augmenter la valeur de k ? Que se passe-t-il lorsqu'on l'augmente trop ? Quelle semble être la valeur donnant le meilleur résultat ?

3 Programmation de l'algorithme

3.1 Mise en place

1. Chargez de nouveau les paramètres d'exemple **Apprentissage supervisé - KNN Caméra** afin de revenir dans la configuration de départ.
2. Nous allons de nouveau utiliser le mode **code python**. Créez un nouveau fichier python que vous pouvez appeler *knn.py*.

3.2 Fonctions auxiliaires

Dans un premier temps, nous allons définir des fonctions auxiliaires qui nous serviront pour l'algorithme final. Vous pouvez définir ces fonctions dans le fichier *knn.py*, en-dessous de la fonction `take_decision`. On rappelle que la distance euclidienne d entre deux points A et B de coordonnées (x_A, y_A) et (x_B, y_B) est :

$$d = \sqrt{(x_B - x_A)^2 + (y_B - y_A)^2}$$

1. Définir une fonction `distance` qui prend en paramètres deux tableaux (ou listes) `a` et `b` contenant chacun deux nombres représentant les coordonnées de deux points A et B, et dont la valeur de retour est la distance entre ces deux points. Le **prototype** de la fonction `distance` doit être : `(List[float], List[float]) -> float`.
2. Testez votre fonction `distance` à l'aide des lignes suivantes (copiez ces lignes en bas de votre fichier python et observez le résultat dans la console de AlphAI). Vérifiez que la valeur affichée est correcte.

```
a = [0, 0]
b = [1, 2]
print("La distance entre", a, "et", b, "est", distance(a, b))
```

3. En utilisant la fonction `distance`, définir une fonction `all_distances` qui prend en paramètres un point `a` (c'est-à-dire une liste de taille 2) et une liste de points `point_list`, et qui renvoie la liste des distances entre le point A et chacun des éléments de la liste `point_list`. Le **prototype** de la fonction `all_distance` doit être : `(List[float], List[List[float]]) -> List[float]`.
4. Testez votre fonction `all_distances` à l'aide des lignes suivantes (et vérifiez que la valeur affichée est correcte) :

```
a = [0.4, 0.7]
train_sensors = [[0, 0], [0, 1], [1, 0], [1, 1]]
distances_list = all_distances(a, train_sensors)
print("Liste des distances :", distances_list)
```

5. Définir une fonction `find_minimum` qui prend en paramètre une liste de nombres et qui renvoie l'**indice du plus petit élément** de la liste. Son **prototype** doit être : `(list[float]) -> int`. Attention au cas où la liste est vide.
6. Testez votre fonction `find_minimum` à l'aide de la ligne suivante (et vérifiez que la valeur affichée est correcte) :

```
print("Indice du minimum :", find_minimum(distances_list))
```

3.3 Fonction de décision

Maintenant que nous avons défini toutes les fonctions auxiliaires nécessaires, nous allons pouvoir créer la fonction `nearest_neighbor_decision` qui accepte 3 paramètres : `train_sensors`, une liste de points représentant les données d'entraînement du robot ; `train_decisions`, une liste d'entiers représentant les actions associées à chacun des points de `train_sensors` ; et un point `a`, représentant les valeurs des capteurs (les 2 valeurs de luminosité fournies par la caméra). On rappelle qu'un point est représenté par une liste de deux nombres décimaux qui sont ses coordonnées. Sa valeur de retour doit être l'indice de l'action choisie par l'algorithme de décision. La fonction `nearest_neighbor_decision` doit donc avoir pour prototype :
`(train_sensors: List[List[float]], train_decisions: List[int], a: List[float]) -> int`.

1. Définir en 3 lignes la fonction `nearest_neighbor_decision`, selon les 3 étapes suivantes (pensez à utiliser les fonctions auxiliaires définies précédemment) :
   - Calculer les distances entre le point `a` et chacun des points de la liste `train_sensors`.
   - Trouver l'indice de la plus petite de ces distances.
   - Renvoyer l'action correspondant au point d'entrainement le plus proche du point `a`.
2. Testez votre fonction `nearest_neighbor_decision` à l'aide des lignes suivantes (et vérifiez que la valeur affichée est correcte) :

```
train_decisions = [0, 2, 0, 1]
decision = nearest_neighbor_decision(
    train_sensors, train_decisions, a)
print("Décision :", decision)
```

3.4 Utiliser les données mémorisées par le robot

1. Au début du fichier *knn.py* (après les lignes `import`), définissez deux variables `train_sensors` et `train_decisions` dont la valeur initiale est la liste vide `[]`. Ces variables définies à l'extérieur de toute fonction sont appelées variables **globales**. Il est possible de modifier une telle variable globale à l'intérieur d'une fonction grâce au mot-clé `global`, comme dans l'exemple ci-dessous.
2. Recopiez la version de la fonction `learn` ci-dessous, qui permet de stocker les données `X_train` et `y_train` mémorisées par le robot dans les variables `train_sensors` et `train_decisions` afin de pouvoir les utiliser dans la fonction `take_decision`.

```
def learn(X_train, y_train):
    global train_sensors, train_decisions
    train_sensors = X_train
    train_decisions = y_train
    return 0
```

3. Modifiez la fonction `take_decision` afin que le robot prenne des décisions selon l'algorithme du plus proche voisin (utilisez votre fonction `nearest_neighbor_decison`). Dans le cas où les les variables `train_sensors` et `train_decisions` sont vides, renvoyer par défaut la valeur 0 (tourner à gauche).
4. Réalisez un apprentissage du robot et vérifiez qu'il se comporte bien. Vous pouvez utiliser la coloration de l'arrière-plan.

3.5 Utiliser des images complètes

Jusqu'à présent, nous avons utilisé l'algorithme des k plus proches voisins avec des données capteurs qui ne comportaient que deux valeurs (la luminosité à gauche et à droite). C'est ce qui nous a permis de visualiser l'espace des états en deux dimensions. Nous allons maintenant programmer ce même algorithme avec des images complètes et en couleur. Dans cette situation, il ne sera malheureusement pas possible de visualiser l'espace d'état car il comporte plus de deux dimensions, mais les principes de l'apprentissage supervisé et de l'algorithme des k plus proches voisins restent les mêmes.

1. Améliorez votre fonction `distance` afin de pouvoir calculer une distance entre deux images de mêmes dimensions. Appliquez pour cela le principe de la distance euclidienne : il faut prendre la racine carré de la somme des carrés des différences entre les coordonnées (qui sont ici les valeurs des pixels).
2. Dans **AlphAI**, dans l'onglet **Capteurs**, sélectionnez une résolution de caméra `16x12`, et choisissez le mode couleur **RVB** (Rouge-Vert-Bleu). Vérifiez

que votre programme permet à votre robot d'apprendre à partir d'images complètes.

3. L'utilisation d'images complètes permet d'apprendre au robot à faire des tours d'arènes de course. Parvenez-vous à obtenir un comportement satisfaisant ?

3.6 Modifier k, le nombre de voisins

Jusqu'à présent, nous avons programmé l'algorithme des k plus proches voisins avec $k = 1$. Pour pouvoir modifier la valeur de k, il va falloir améliorer plusieurs des fonctions que nous avons programmées.

1. Définir une fonction `find_k_minima`, qui prend en paramètres une liste de nombres `numbers_list` et un entier `k` et qui renvoie une liste contenant les indices des `k` plus petits éléments de la liste. Dans le cas où la liste contient moins de `k` éléments, on pourra renvoyer les indices de tous ces éléments (et ainsi renvoyer une liste de moins de `k` éléments au total). **Conseil** : créer une nouvelle liste contenant des couples `(valeur, indice)`, puis trier cette liste.
2. Définir une fonction `knn` sur le modèle de `nearest_neighbor_decision`, en utilisant cette fois la fonction `find_k_minima`. La variable k pourra être définie et par la suite modifiée dans le code de cette fonction `knn`. Modifiez également la fonction `take_decision` afin d'utiliser cette nouvelle fonction `knn`.
3. Vérifiez que votre programme fonctionne. Pour cela, commencez par utiliser les paramètres de la caméra permettant d'avoir la visualisation en 2D (essayez différentes résolutions et différents modes de traitement d'image, parvenez-vous à retrouver les paramètres utilisés dans le scénario KNN ?) Vérifiez que le changement du paramètre k affecte la coloration de l'arrière-plan.
4. Enfin, vous pouvez paramétrer la caméra afin d'utiliser des images entières et en couleurs afin d'entraîner votre robot à faire des tours de l'arène de course. Modifiez dans votre programme la valeur de k. Quelle valeur de k semble donner le meilleur comportement ? Comment évaluer la performance de l'algorithme en l'absence de visualisation ?